Machine Intelligence 1

1.7 Radial Basis Function Networks

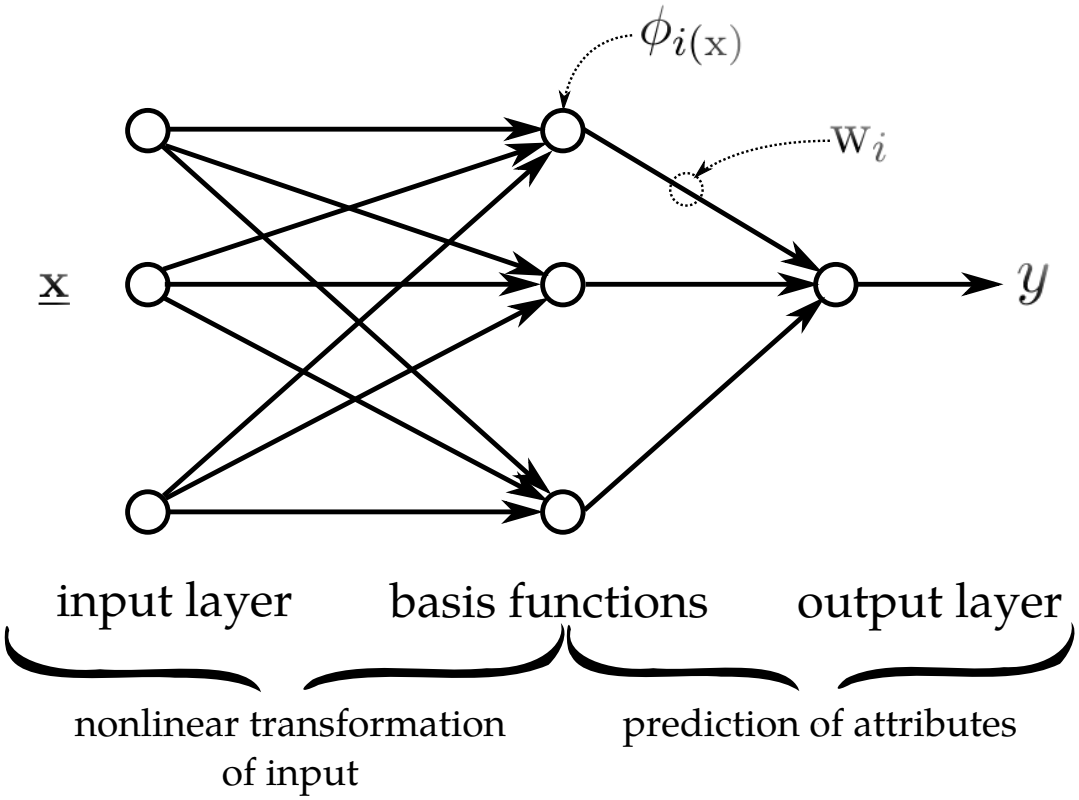
Prof. Dr. Klaus Obermayer

Fachgebiet Neuronale Informationsverarbeitung (NI)

WS 25/26

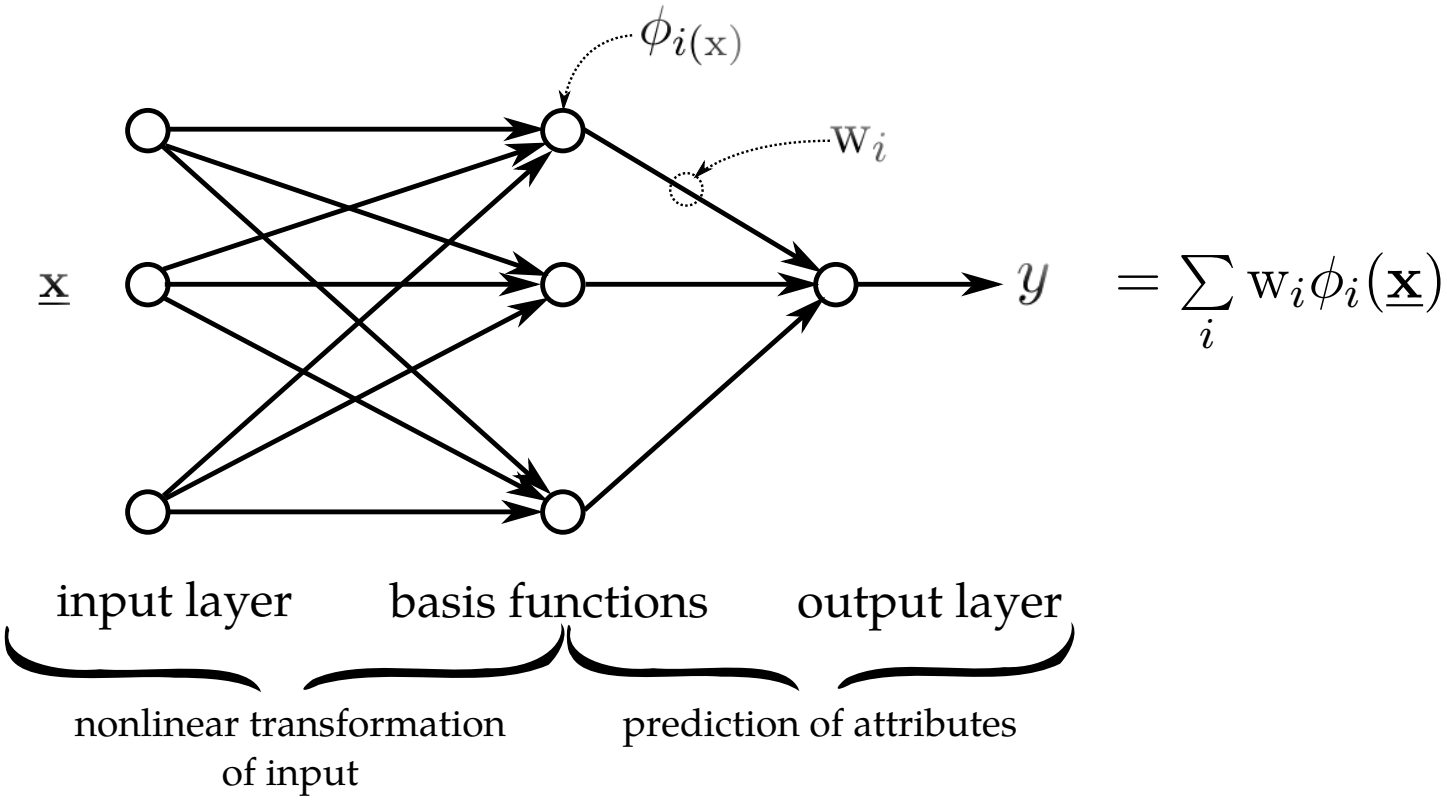
1.7.1 Network Architecture

Network architecture



$$= \sum_i \mathrm{w}_i \phi_i(\underline{\mathbf{x}})$$

Network architecture



General principle

- two layered network $\rightarrow$ expansion into basis functions / features
- sine-waves (Fourier), polynomials (Taylor), sigmoid functions (MLP)

Radial basis functions (RBF)

- **Distance dependent basis functions**

$$\phi_i(\underline{\mathbf{x}}) \quad = \quad \widetilde{\phi}_i(D[\underline{\mathbf{x}}, \underline{\mathbf{t}}_i])$$

where $\underline{\mathbf{t}}_i$ are parameters specifying the location of the i-th basis function

Radial basis functions (RBF)

- **Distance dependent basis functions**

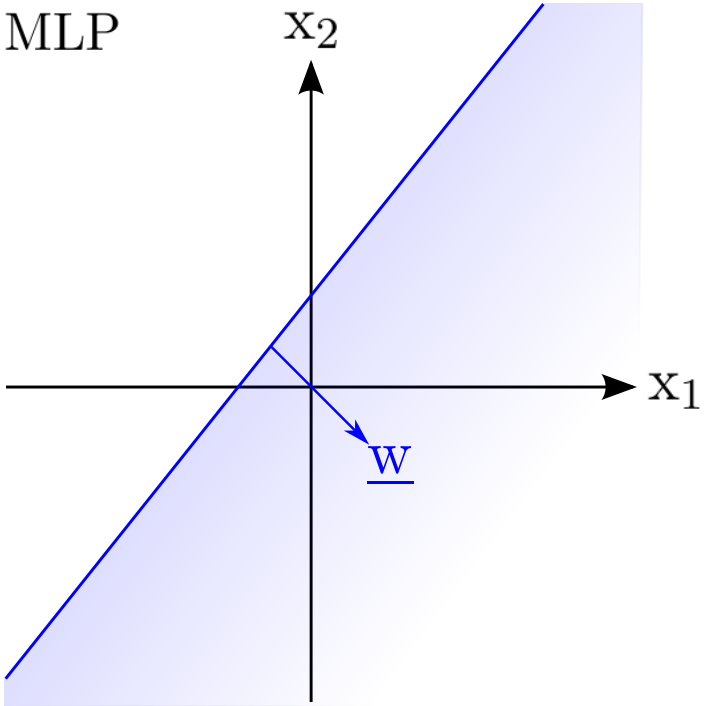
$$\phi_i(\underline{\mathbf{x}}) \quad = \quad \widetilde{\phi}_i(D[\underline{\mathbf{x}}, \underline{\mathbf{t}}_i])$$

where $\underline{\mathbf{t}}_i$ are parameters specifying the location of the i-th basis function

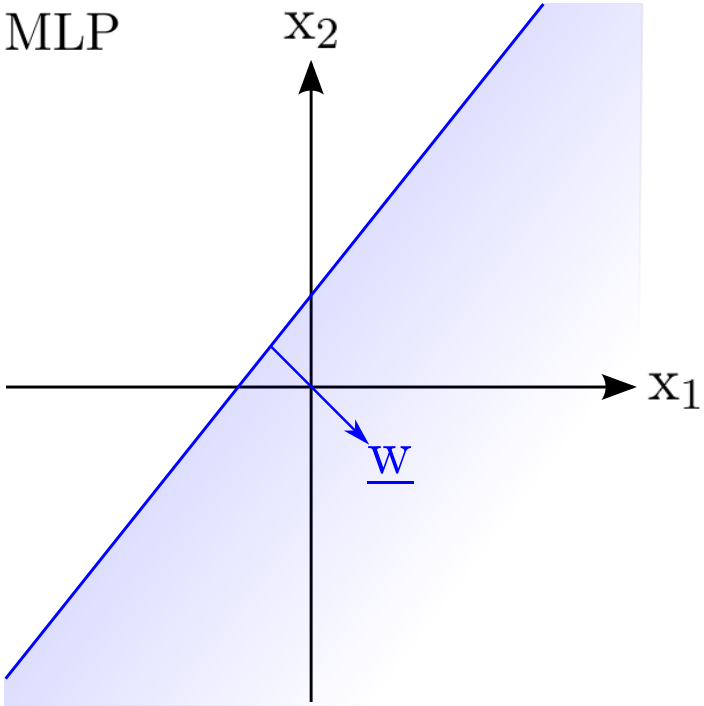
- **Common choice:** Gaussian functions

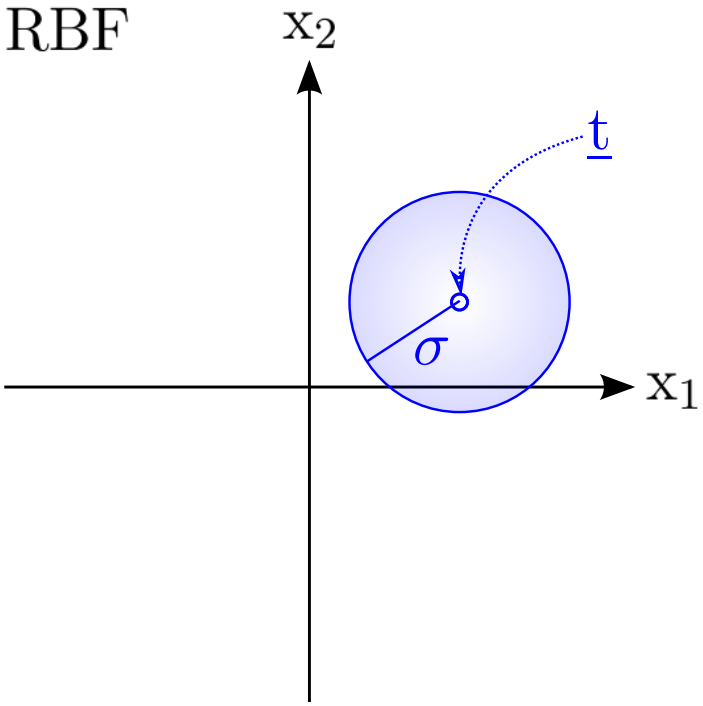
$$\phi_i(\underline{\mathbf{x}}) \quad \propto \quad \exp\left(- \frac{\|\underline{\mathbf{x}} - \underline{\mathbf{t}}_i\|^2}{2\sigma_i^2} \right)$$

MLP vs. RBF

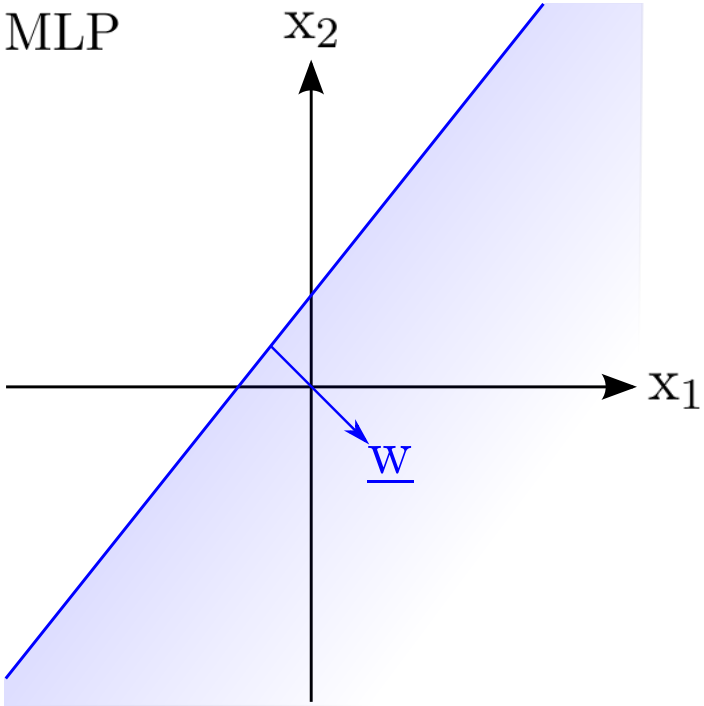


MLP vs. RBF

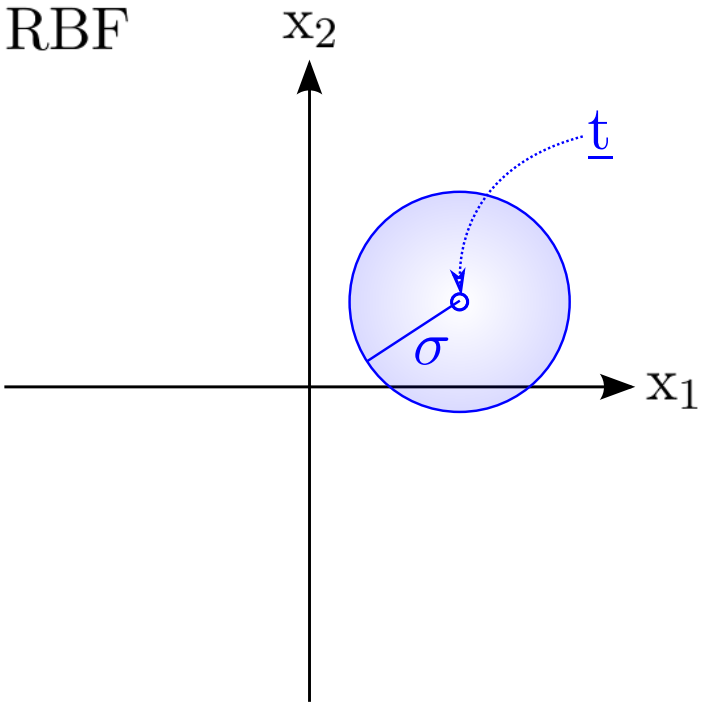




MLP vs. RBF

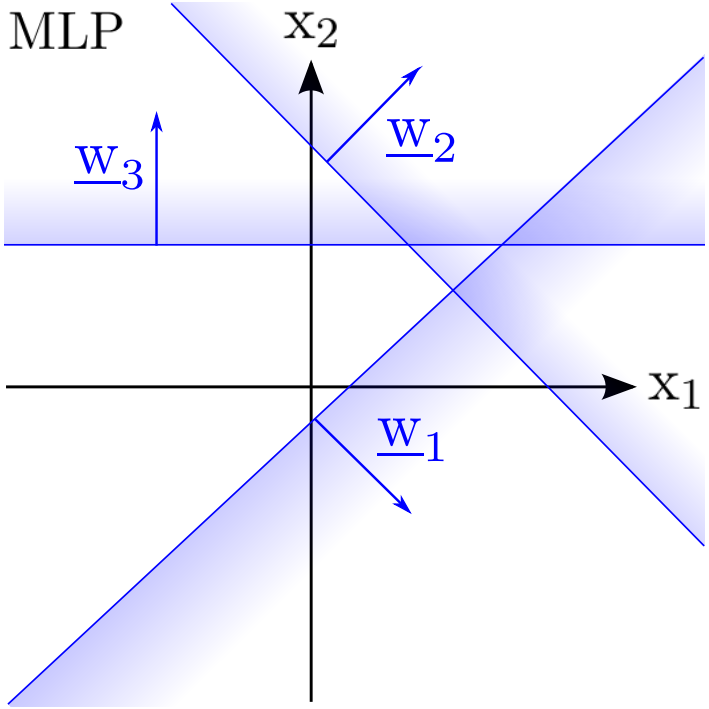


$\Rightarrow$ "discriminative features"

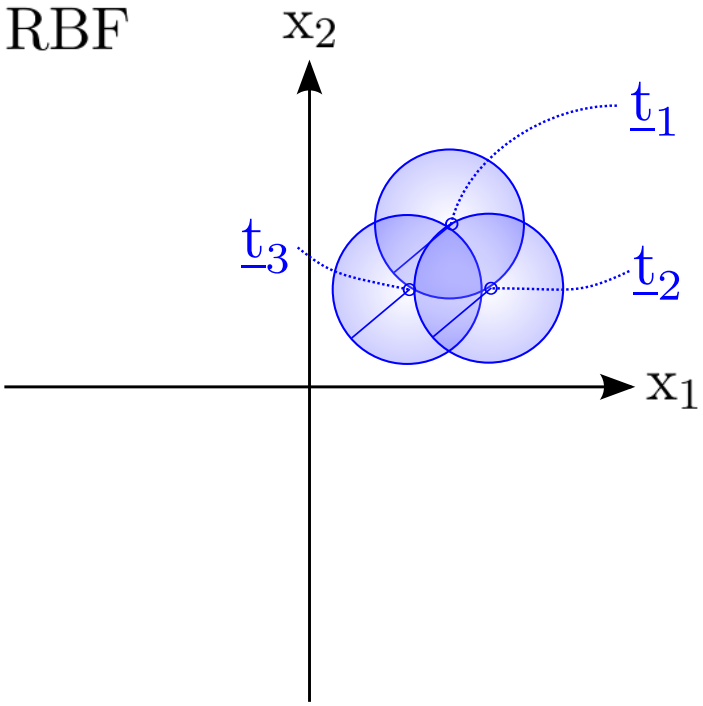


$\Rightarrow$ "categories/classes"

MLP vs. RBF



$\Rightarrow$ "global" approximation



$\Rightarrow$ "local" approximation

RBFs: pro & contra

fast convergence during learning

→ few parameters have to be changed per training point

→ "credit assignment" is simple

RBFs: pro & contra

fast convergence during learning

$\rightarrow$ few parameters have to be changed per training point

$\rightarrow$ "credit assignment" is simple

"curse of dimensionality"

complete coverage of input space requires $\sim M^N$ basis functions
(N: dimension, M: no. of basis functions along one dimension)
$N = 20, M = 10 \rightsquigarrow 10^{20}$ basis functions

RBFs: pro & contra

fast convergence during learning

$\rightarrow$ few parameters have to be changed per training point

$\rightarrow$ "credit assignment" is simple

"curse of dimensionality"

complete coverage of input space requires $\sim M^N$ basis functions
(N: dimension, M: no. of basis functions along one dimension)
$N = 20, M = 10 \rightsquigarrow 10^{20}$ basis functions

$\Rightarrow$ **RBF-networks are useful for**

- low dimensional data or
- datasets with a pronounced cluster structure

1.7.2 Model Selection – Learning

Problem setting & model class

Regression: real-valued targets

$$\left\{\left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\right)\right\}, \quad \alpha \in \{1, \ldots, p\}, \qquad \underline{\mathbf{x}} \in \mathbb{R}^N, \quad y_T \in \mathbb{R}$$

Problem setting & model class

Regression: real-valued targets

$$\left\{\left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\right)\right\}, \quad \alpha \in \{1, \ldots, p\}, \qquad \underline{\mathbf{x}} \in \mathbb{R}^N, \quad y_T \in \mathbb{R}$$

Model class:

$$y_{(\underline{\mathbf{x}})} = \sum_{i=1}^{M} \mathrm{w}_i \exp\left(-\frac{\|\underline{\mathbf{x}} - \underline{\mathbf{t}}_i\|^2}{2\sigma_i^2}\right)$$

1. $\underline{\mathbf{t}}_i$: centroids of basis functions
2. σ_i: range of basis functions
3. w_i: weights of the output layer

Model selection / learning

2-Step learning procedure:

1. $\underline{\mathbf{t}}_i$: centroids — unsupervised
2. σ_i: ranges — heuristics
3. w_i: weights — supervised

Model selection / learning

2-Step learning procedure:

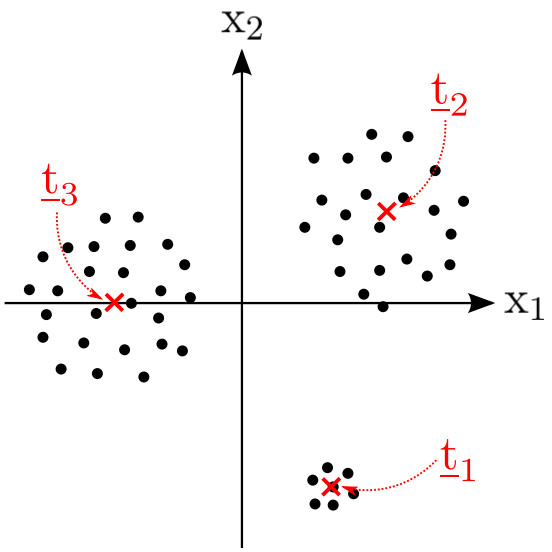
1. $\underline{\mathbf{t}}_i$: centroids — unsupervised
2. σ_i: ranges — heuristics
3. w_i: weights — supervised

Alternative: supervised learning of all parameters

But: non-convex problem with local minima

Determination of centroids $\underline{\mathbf{t}}_i$

k-means clustering (online)



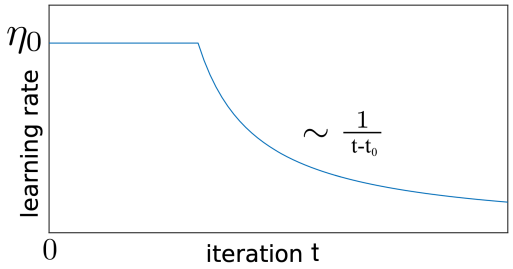
Initialize $\underline{\mathbf{t}}_i$

BEGIN loop

1. Choose data point $\underline{\mathbf{x}}^{(\alpha)}$
2. Closest centroid $\underline{\mathbf{t}}_i$: $i = \mathrm{argmin}_j \left| \underline{\mathbf{t}}_j - \underline{\mathbf{x}}^{(\alpha)} \right|$
3. Update $\underline{\mathbf{t}}_i$ as: $\Delta \underline{\mathbf{t}}_i = \eta_t \left(\underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{t}}_i \right)$

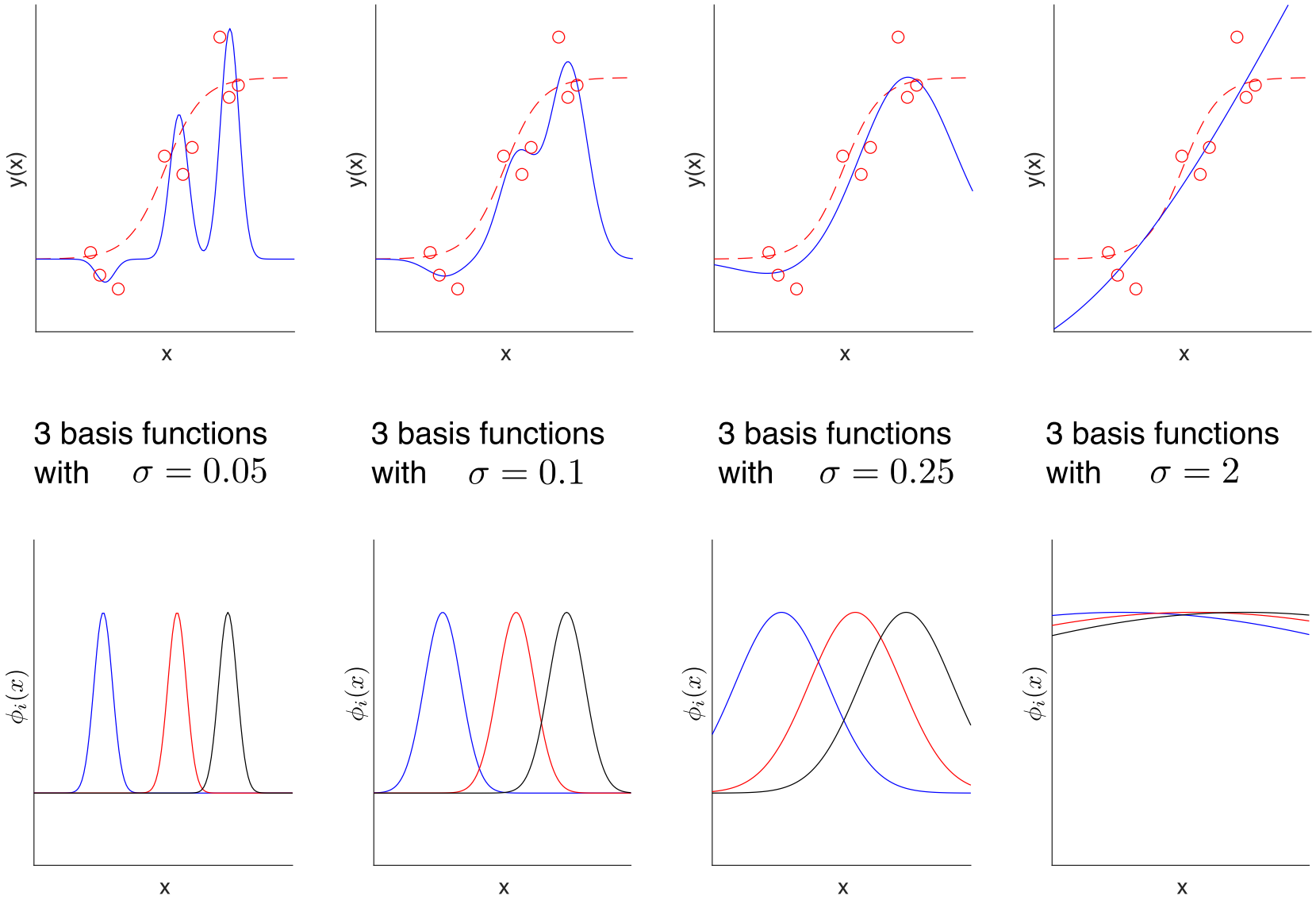
END loop

- Adaptive learning rate η_t[1]
  - $\eta_t = \eta_0$ for $t \leq t_0$
  - $\eta_t = \frac{\eta_0}{t - t_0}$ for $t > t_0$



[1] The learning step can be made centroid-dependent: $\eta_t^{(i)} \propto \frac{1}{n_t^{(i)}}$, where $n_t^{(i)}$ is the number of past updates of prototype $\underline{\mathbf{t}}_i$ up to iteration t.

Determination of widths σ_i



3 basis functions with $\sigma = 0.05$

3 basis functions with $\sigma = 0.1$

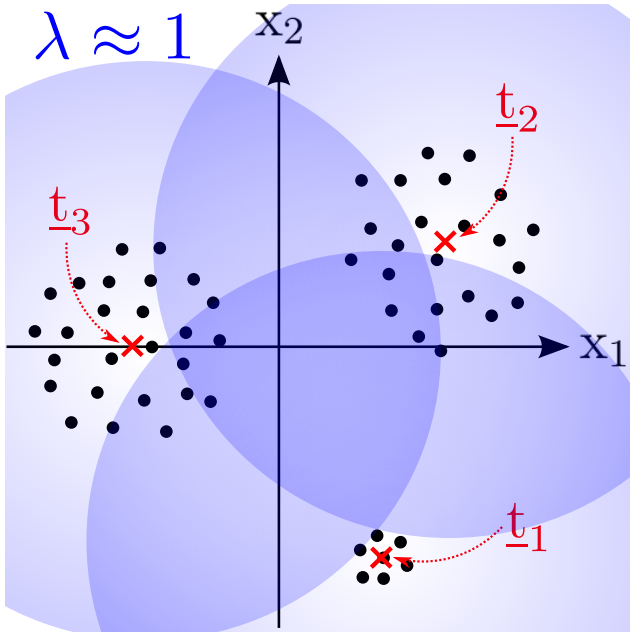
3 basis functions with $\sigma = 0.25$

3 basis functions with $\sigma = 2$

Determination of variances σ_i

Goal: sufficient overlap between neighboring basis functions

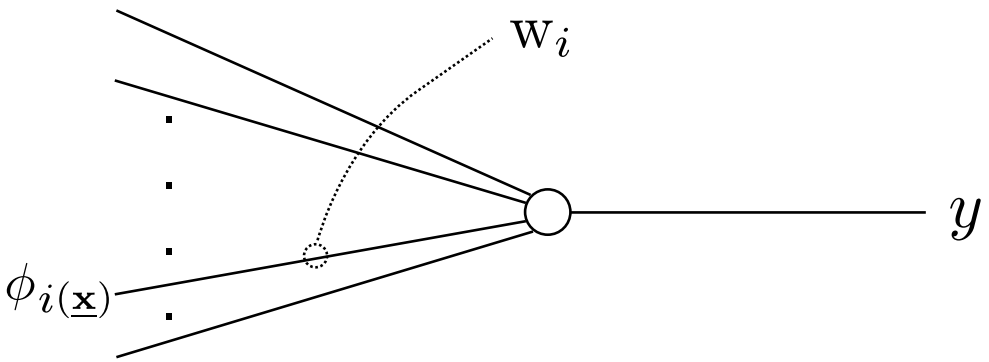
Problem: coverage vs. resolution payoff



Heuristic

$$\sigma_i \quad = \quad \lambda \min_{j \neq i} \left\| \underline{\mathbf{t}}_i - \underline{\mathbf{t}}_j \right\|, \qquad \lambda \approx 2$$

Determination of output weights w_i



(linear)
connectionist neuron

Cost function: quadratic error

$$E^T = \frac{1}{2p} \sum_{\alpha=1}^{p} \left(y_T^{(\alpha)} - \sum_{i=1}^{M} \mathrm{w}_i \phi_i(\underline{\mathbf{x}}^{(\alpha)}) \right)^2$$

Optimization (1)

$$\frac{\partial E^T}{\partial \mathrm{w}_k} = \frac{\partial}{\partial \mathrm{w}_k} \cdot \frac{1}{2p} \sum_{\alpha=1}^{p} \left(y_T^{(\alpha)} - \sum_{i=1}^{M} \mathrm{w}_i \underbrace{\phi_i(\underline{\mathbf{x}}^{(\alpha)})}_{:=\phi_i^{(\alpha)}} \right)^2$$

$$= -\frac{1}{p} \sum_{\alpha=1}^{p} \left(y_T^{(\alpha)} - \sum_{i=1}^{M} \mathrm{w}_i \phi_i^{(\alpha)} \right) \phi_k^{(\alpha)} \stackrel{!}{=} 0$$

Optimization (1)

$$\frac{\partial E^T}{\partial \mathrm{w}_k} = \frac{\partial}{\partial \mathrm{w}_k} \cdot \frac{1}{2p} \sum_{\alpha=1}^{p} \left(y_T^{(\alpha)} - \sum_{i=1}^{M} \mathrm{w}_i \underbrace{\phi_i(\underline{\mathbf{x}}^{(\alpha)})}_{:=\phi_i^{(\alpha)}} \right)^2$$

$$= -\frac{1}{p} \sum_{\alpha=1}^{p} \left(y_T^{(\alpha)} - \sum_{i=1}^{M} \mathrm{w}_i \phi_i^{(\alpha)} \right) \phi_k^{(\alpha)} \stackrel{!}{=} 0$$

$$\sum_{i=1}^{M} \left(\sum_{\alpha=1}^{p} \phi_k^{(\alpha)} \phi_i^{(\alpha)} \right) \mathrm{w}_i = \sum_{\alpha=1}^{p} \phi_k^{(\alpha)} y_T^{(\alpha)}$$

Optimization (1)

$$\frac{\partial E^T}{\partial \mathrm{w}_k} = \frac{\partial}{\partial \mathrm{w}_k} \cdot \frac{1}{2p} \sum_{\alpha=1}^{p} \left(y_T^{(\alpha)} - \sum_{i=1}^{M} \mathrm{w}_i \underbrace{\phi_i(\underline{\mathbf{x}}^{(\alpha)})}_{:=\phi_i^{(\alpha)}} \right)^2$$

$$= -\frac{1}{p} \sum_{\alpha=1}^{p} \left(y_T^{(\alpha)} - \sum_{i=1}^{M} \mathrm{w}_i \phi_i^{(\alpha)} \right) \phi_k^{(\alpha)} \stackrel{!}{=} 0$$

$$\sum_{i=1}^{M} \left(\sum_{\alpha=1}^{p} \phi_k^{(\alpha)} \phi_i^{(\alpha)} \right) \mathrm{w}_i = \sum_{\alpha=1}^{p} \phi_k^{(\alpha)} y_T^{(\alpha)}$$

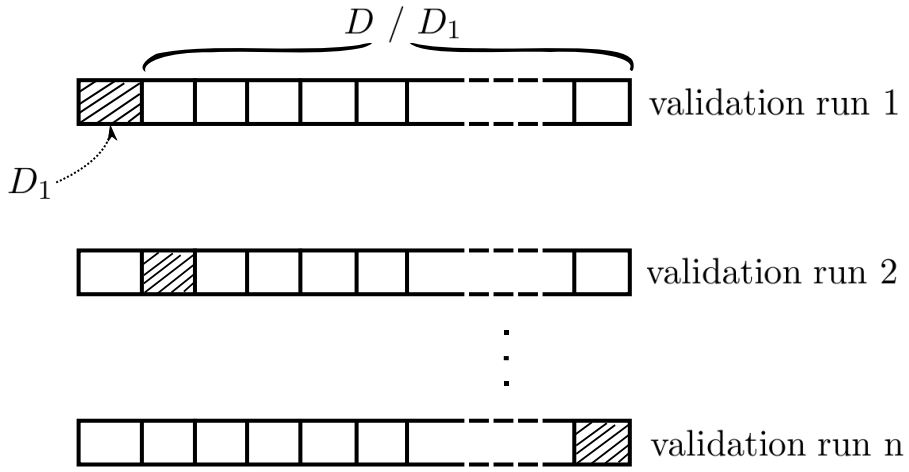
Matrix notation

$$\underbrace{\left(\underline{\mathbf{\Phi}}^\top \underline{\mathbf{\Phi}}\right)}_{\text{known}} \underline{\mathbf{w}} = \underbrace{\underline{\mathbf{\Phi}}^\top \underline{\mathbf{y}}_T}_{\text{known}} \quad \Rightarrow \quad \underline{\mathbf{w}} = \underbrace{\left(\underline{\mathbf{\Phi}}^\top \underline{\mathbf{\Phi}}\right)^{-1}}_{\text{if invertible}} \underline{\mathbf{\Phi}}^\top \underline{\mathbf{y}}_T$$

$$\begin{aligned} \underline{\mathbf{\Phi}} &= \{\phi_k^{(\alpha)}\} && p \times M \text{ matrix} \\ \underline{\mathbf{w}} &= \{\mathrm{w}_i\} && M \text{ vector} \\ \underline{\mathbf{y}}_T &= \{y_T^{(\alpha)}\} && p \text{ vector} \end{aligned}$$

Validation

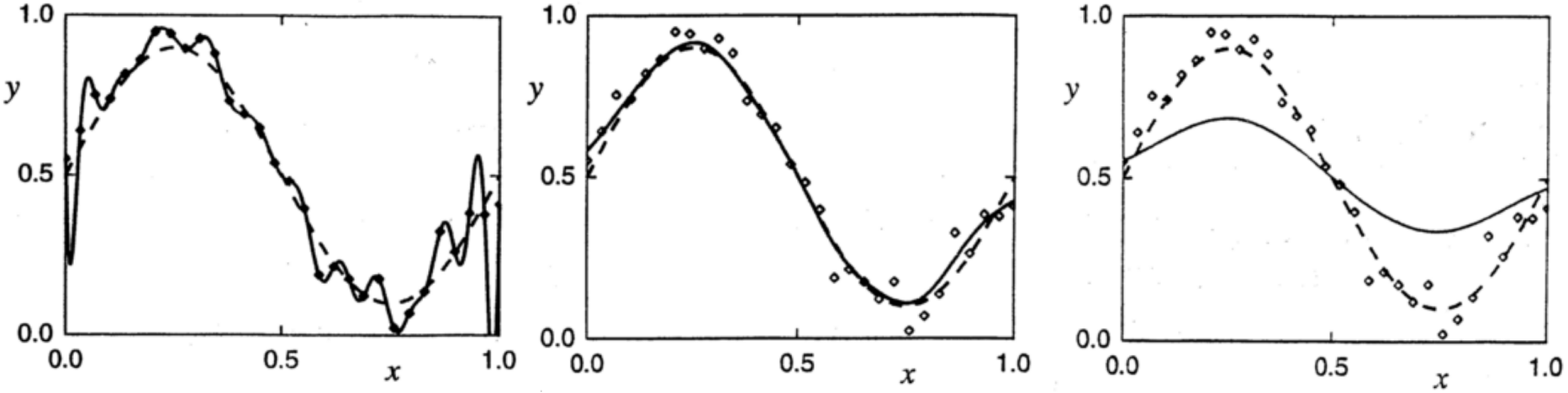
- test-set method
- n-fold cross-validation



training on D/D_i must include all three model selection steps

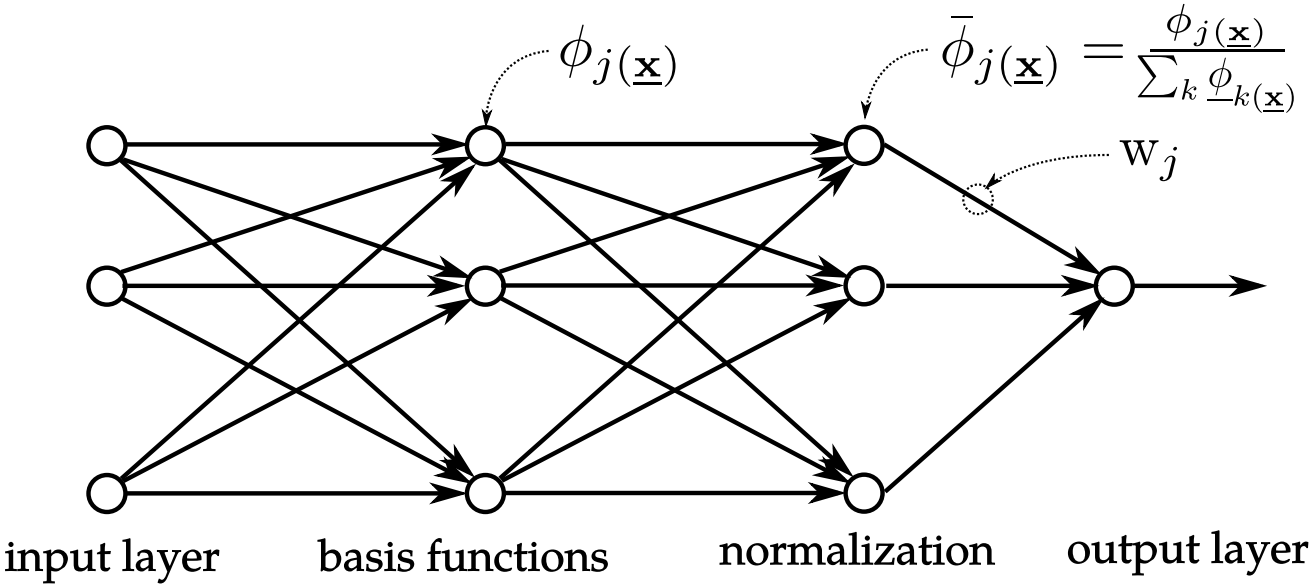
- use n-fold cross-validation to determine other model parameters (e.g. number of basis functions)

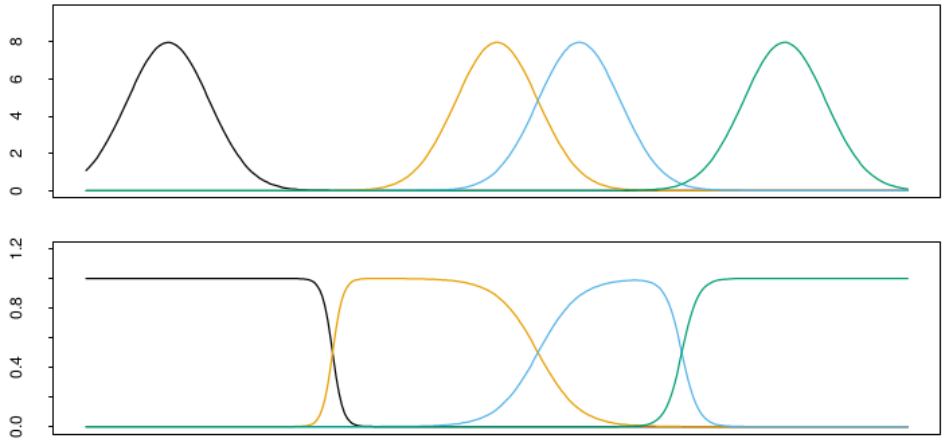
Comment: number of basis functions



- too many basis functions ⇒ overfitting
- too few basis functions ⇒ underfitting

Comment: normalization layer





Comment: regularization

- Matrix $\underline{\mathbf{\Phi}}^\top \underline{\mathbf{\Phi}}$ often not invertible!
- Add *ridge regression* term $\lambda E^R_{[\underline{\mathbf{w}}]} = \lambda \|\underline{\mathbf{w}}\|^2$ to cost function
  - regularized solution $\underline{\mathbf{w}} = \underbrace{\left(\underline{\mathbf{\Phi}}^\top \underline{\mathbf{\Phi}} + \lambda \underline{\mathbf{I}}\right)^{-1}}_{\text{invertible for } \lambda > 0} \underline{\mathbf{\Phi}}^\top \underline{\mathbf{y}}_T$
  - larger λ yield *smoother* functions

$\Rightarrow$ Use n-fold cross validation to determine λ

Comment: two-step procedure vs. gradient descent

The two-step procedure...

- ...is much faster
- ...has usually equal performance
- ...can use additional unlabeled data

1.7.3 RBF-Networks and Regularization

General model classes

General learning problem

observations: $\left\{\left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\right)\right\}, \quad \alpha \in \{1, \ldots, p\}$

model class: all continuous and differentiable functions $y_{(\underline{\mathbf{x}})}$

cost function: $E^T = \frac{1}{2p} \sum_{\alpha=1}^{p} \left(y(\underline{\mathbf{x}}^{(\alpha)}) - y_T^{(\alpha)}\right)^2$

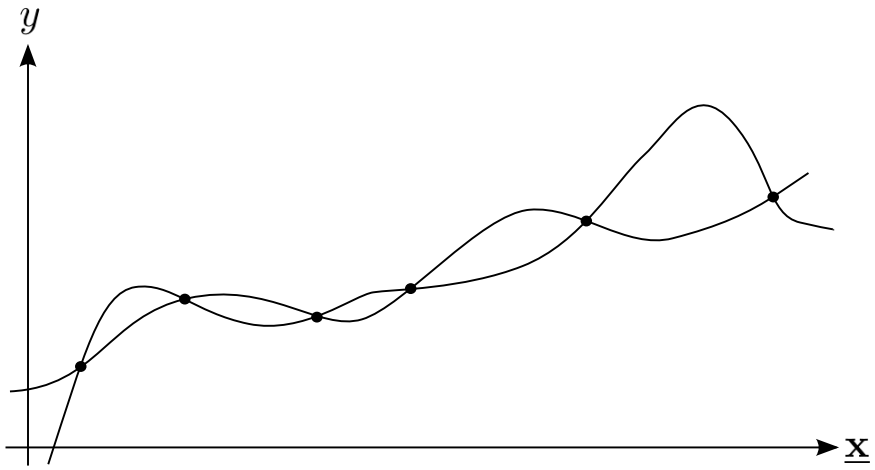
General model classes

General learning problem

observations: $\left\{\left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\right)\right\}, \quad \alpha \in \{1, \ldots, p\}$

model class: all continuous and differentiable functions $y_{(\underline{\mathbf{x}})}$

cost function: $E^T = \frac{1}{2p} \sum_{\alpha=1}^{p} \left(y(\underline{\mathbf{x}}^{(\alpha)}) - y_T^{(\alpha)}\right)^2$



many functions are consistent with the data

$\Rightarrow$ **ill-posed learning problem**

Regularization

New cost function: $R = E^T + \lambda E^R$ (Tikhonov, 1963)

Regularization

New cost function: $R = E^T + \lambda E^R$ (Tikhonov, 1963)

$$y(\underline{\mathbf{x}}) = \int d\underline{\mathbf{k}} \quad e^{(i\underline{\mathbf{k}}^\top \underline{\mathbf{x}})}\, \tilde{y}(\underline{\mathbf{k}}) \qquad \text{(Fourier transform)}$$

Regularization

New cost function: $R = E^T + \lambda E^R$ (Tikhonov, 1963)

$$y(\underline{\mathbf{x}}) = \int d\underline{\mathbf{k}} \quad e^{(i\underline{\mathbf{k}}^\top \underline{\mathbf{x}})}\, \tilde{y}(\underline{\mathbf{k}}) \qquad \text{(Fourier transform)}$$

$$E^R = \frac{1}{2} \int d\underline{\mathbf{k}} \frac{\left|\tilde{y}(\underline{\mathbf{k}})\right|^2}{\widetilde{G}(\underline{\mathbf{k}})} \qquad \text{(regularization)}$$

Regularization

New cost function: $R = E^T + \lambda E^R$ (Tikhonov, 1963)

$$y(\underline{\mathbf{x}}) = \int d\underline{\mathbf{k}} \quad e^{(i\underline{\mathbf{k}}^\top \underline{\mathbf{x}})}\, \tilde{y}(\underline{\mathbf{k}}) \qquad \text{(Fourier transform)}$$

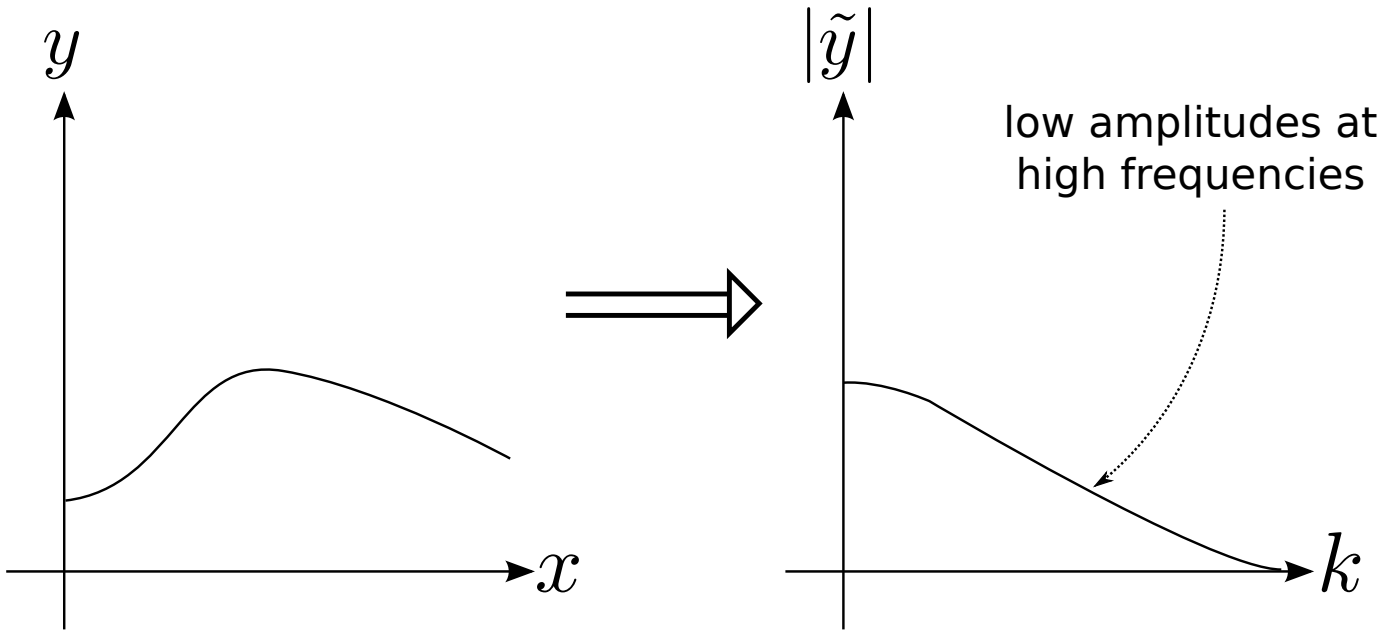
$$E^R = \frac{1}{2} \int d\underline{\mathbf{k}} \frac{\left|\tilde{y}(\underline{\mathbf{k}})\right|^2}{\widetilde{G}(\underline{\mathbf{k}})} \qquad \text{(regularization)}$$

- Filter $\widetilde{G}(\underline{\mathbf{k}})$ imposes (soft-)constraints on $\tilde{y}(k) \rightsquigarrow$ functions $y(x)$.

$\Rightarrow$ **well-posed problem** (existence, uniqueness, continuity, see Haykin, ch. 5)

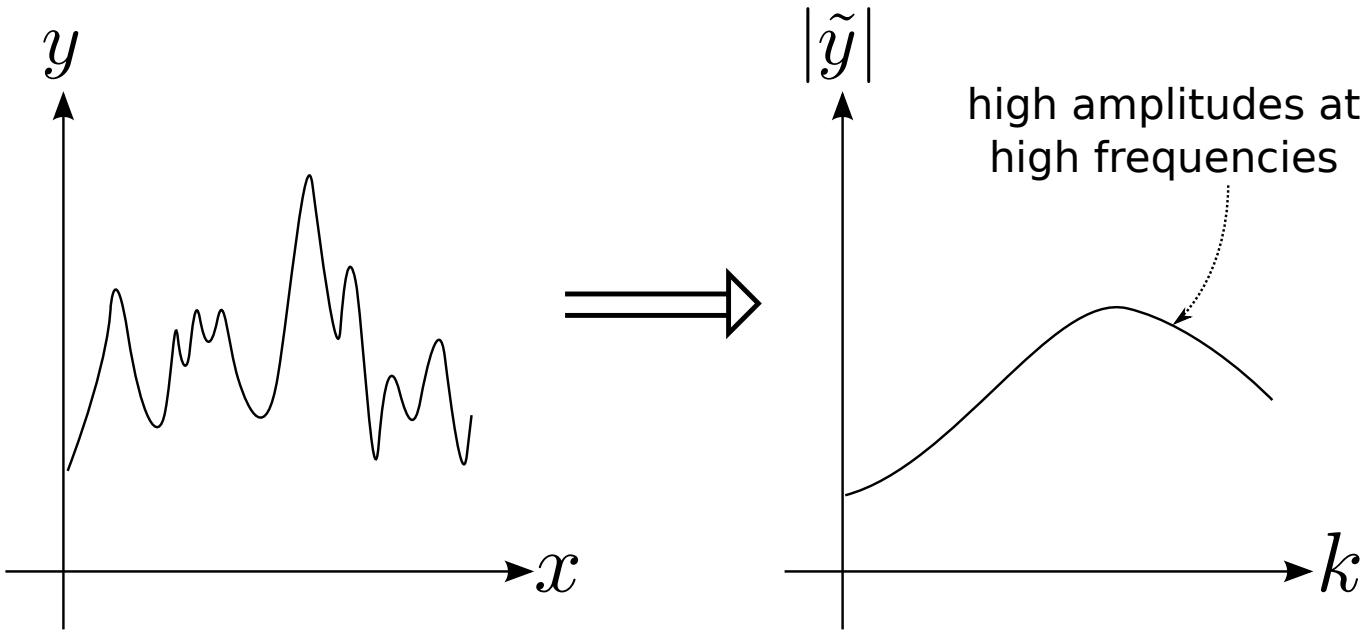
Smooth functions in Fourier space

$$y(\underline{\mathbf{x}}) = \int d\underline{\mathbf{k}} \quad e^{(i\underline{\mathbf{k}}^\top \underline{\mathbf{x}})}\, \tilde{y}(\underline{\mathbf{k}}) \qquad \text{(Fourier transform)}$$



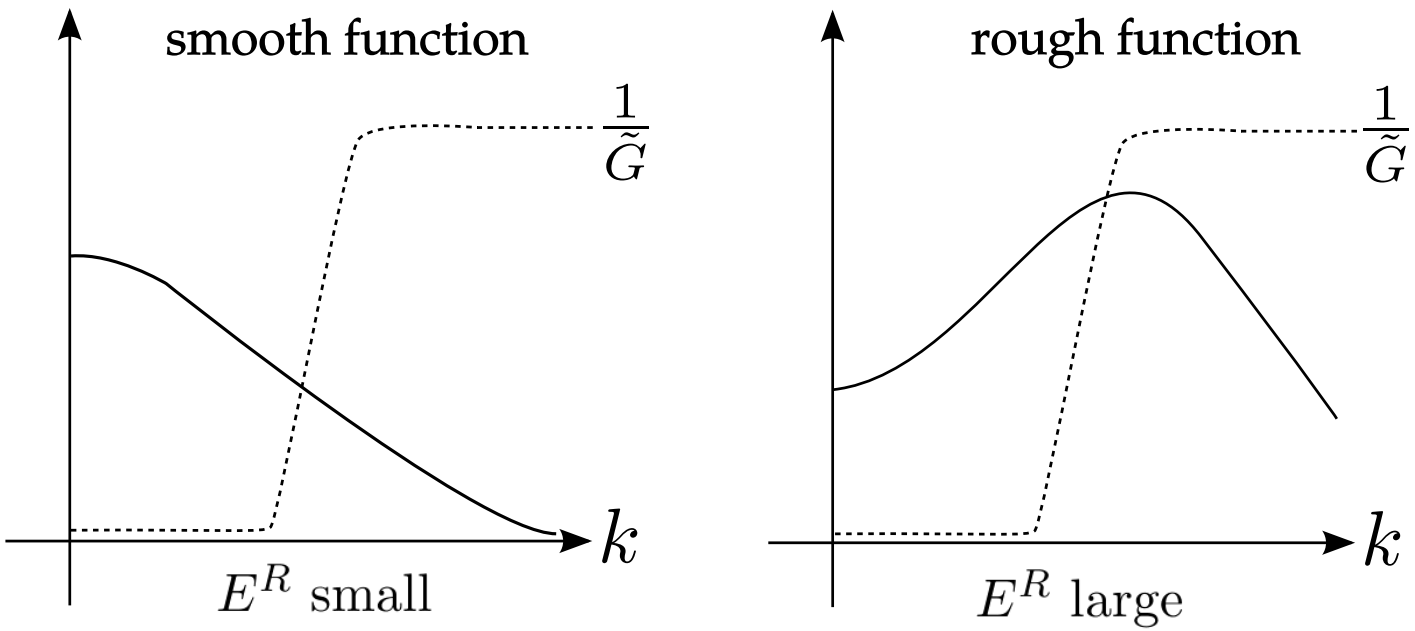
Rough functions in Fourier space

$$y(\underline{\mathbf{x}}) = \int d\underline{\mathbf{k}} \quad e^{(i\underline{\mathbf{k}}^\top \underline{\mathbf{x}})}\, \tilde{y}(\underline{\mathbf{k}}) \qquad \text{(Fourier transform)}$$



Effects of regularization using a smoothness constraint

$$E^R = \frac{1}{2} \int d\underline{\mathbf{k}} \frac{\left|\tilde{y}(\underline{\mathbf{k}})\right|^2}{\tilde{G}(\underline{\mathbf{k}})}$$



Result of model selection

$$R \quad = \quad E^T + \lambda E^R \quad = \quad \min_{y(\underline{\mathbf{x}})}$$

Result of model selection

$$R \quad = \quad E^T + \lambda E^R \quad = \quad \min_{y(\underline{\mathbf{x}})}$$

$$y(\underline{\mathbf{x}}) = \sum_{\alpha=1}^{p} \mathrm{w}_\alpha \, G(\underline{\mathbf{x}} - \underline{\mathbf{x}}^{(\alpha)})$$ (RBF-network depending on filter)

with $$G(\underline{\mathbf{x}}) = \int d\underline{\mathbf{k}} \; e^{(i\underline{\mathbf{k}}^\top \underline{\mathbf{x}})} \, \widetilde{G}(\underline{\mathbf{k}})$$ (Fourier-transform of filter)

Result of model selection

$$R \quad = \quad E^T + \lambda E^R \quad = \quad \min_{y(\underline{\mathbf{x}})}$$

$$y(\underline{\mathbf{x}}) = \sum_{\alpha=1}^{p} \mathrm{w}_\alpha \, G(\underline{\mathbf{x}} - \underline{\mathbf{x}}^{(\alpha)})$$ (RBF-network depending on filter)

with $$G(\underline{\mathbf{x}}) = \int d\underline{\mathbf{k}} \; e^{(i\underline{\mathbf{k}}^\top \underline{\mathbf{x}})} \, \widetilde{G}(\underline{\mathbf{k}})$$ (Fourier-transform of filter)

- prior knowledge determines shape of basis functions
- location of data points determine location of centroids (unsupervised)

Result of model selection

$$R \quad = \quad E^T + \lambda E^R \quad = \quad \min_{y(\underline{\mathbf{x}})}$$

$$y(\underline{\mathbf{x}}) = \sum_{\alpha=1}^{p} \mathrm{w}_\alpha \, G(\underline{\mathbf{x}} - \underline{\mathbf{x}}^{(\alpha)})$$ (RBF-network depending on filter)

with $$G(\underline{\mathbf{x}}) = \int d\underline{\mathbf{k}} \; e^{(i\underline{\mathbf{k}}^\top \underline{\mathbf{x}})} \, \widetilde{G}(\underline{\mathbf{k}})$$ (Fourier-transform of filter)

- prior knowledge determines shape of basis functions
- location of data points determine location of centroids (unsupervised)

$$\underline{\mathbf{w}} = \frac{1}{p\lambda} \underline{\mathbf{G}}^{-1} \left(\underline{\mathbf{G}}^{-1} + \frac{1}{p\lambda} \, \underline{\mathbf{I}} \right)^{-1} \underline{\mathbf{y}}_T \,, \quad \text{where} \quad G_{\alpha\beta} = G(\underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{x}}^{(\beta)})$$

- solution equivalent to ridge regression

Example: Gaussian filters

Prior on smooth functions: penalize high frequencies

Example: Gaussian filters

Prior on smooth functions: penalize high frequencies

$$E^R = \frac{1}{2}\int d\underline{\mathbf{k}} \frac{|\tilde{y}(\underline{\mathbf{k}})|^2}{\widetilde{G}(\underline{\mathbf{k}})} = \int d\underline{\mathbf{k}} \; \underbrace{e^{\sigma^2 \underline{\mathbf{k}}^2}}_{\text{high pass}} \; |\tilde{y}_{(\underline{\mathbf{k}})}|^2$$

Example: Gaussian filters

Prior on smooth functions: penalize high frequencies

$$E^R = \frac{1}{2}\int d\underline{\mathbf{k}} \frac{|\tilde{y}(\underline{\mathbf{k}})|^2}{\widetilde{G}(\underline{\mathbf{k}})} = \int d\underline{\mathbf{k}} \underbrace{e^{\sigma^2 \underline{\mathbf{k}}^2}}_{\text{high pass}} \left|\tilde{y}_{(\underline{\mathbf{k}})}\right|^2$$

leads to:

$$G(\underline{\mathbf{x}}) \sim \exp\left(-\frac{\underline{\mathbf{x}}^2}{\sigma^2}\right)$$

Example: Gaussian filters

Prior on smooth functions: penalize high frequencies

$$E^R = \frac{1}{2}\int d\underline{\mathbf{k}} \frac{|\tilde{y}(\underline{\mathbf{k}})|^2}{\widetilde{G}(\underline{\mathbf{k}})} = \int d\underline{\mathbf{k}} \underbrace{e^{\sigma^2 \underline{\mathbf{k}}^2}}_{\text{high pass}} \left|\tilde{y}_{(\underline{\mathbf{k}})}\right|^2$$

leads to:

$$G(\underline{\mathbf{x}}) \sim \exp\left(-\frac{\underline{\mathbf{x}}^2}{\sigma^2}\right)$$

$\Rightarrow$ close connection between RBF-networks and regularization

$\Rightarrow$ yet another model selection procedure for RBF-networks

Example: Gaussian filters

Prior on smooth functions: penalize high frequencies

$$E^R = \frac{1}{2}\int d\underline{\mathbf{k}} \frac{|\tilde{y}(\underline{\mathbf{k}})|^2}{\widetilde{G}(\underline{\mathbf{k}})} = \int d\underline{\mathbf{k}} \underbrace{e^{\sigma^2 \underline{\mathbf{k}}^2}}_{\text{high pass}} \left|\tilde{y}_{(\underline{\mathbf{k}})}\right|^2$$

leads to:

$$G(\underline{\mathbf{x}}) \sim \exp\left(-\frac{\underline{\mathbf{x}}^2}{\sigma^2}\right)$$

$\Rightarrow$ close connection between RBF-networks and regularization

$\Rightarrow$ yet another model selection procedure for RBF-networks

Problem: Number of basis functions = no. of data points (large!)

$\rightsquigarrow$ sparse expansion desirable

$\rightsquigarrow$ support vector machines

References I

Simon Haykin. *Neural networks: a comprehensive foundation*. Prentice Hall PTR, 1994.

A. N. Tikhonov. Solution of incorrectly formulated problems and the regularization method. *Soviet Math. Dokl.*, 4:1035–1038, 1963.

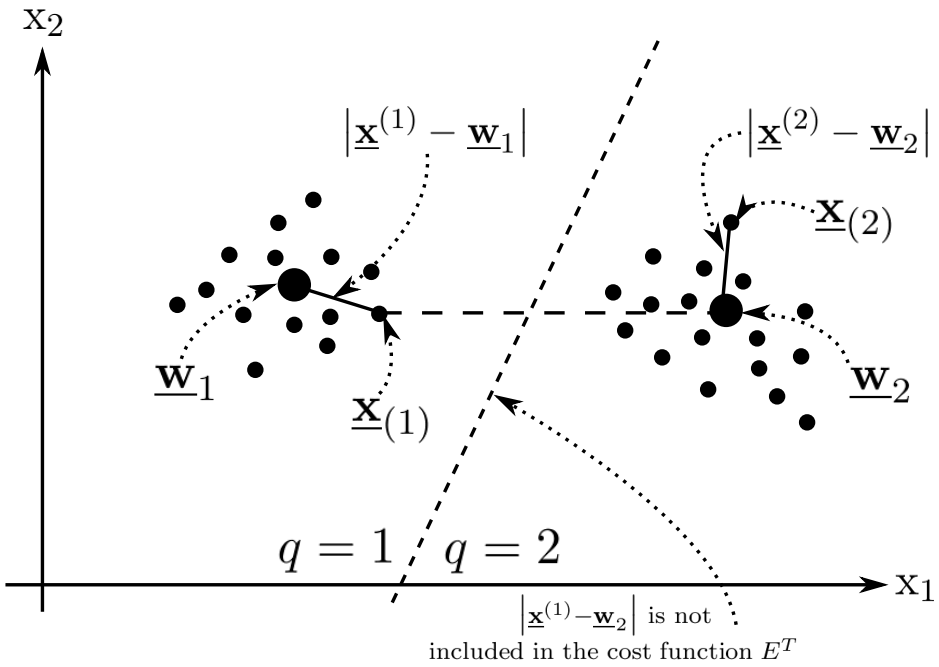
End of Section 1.5

the following slides contain

OPTIONAL MATERIAL

Batch K-Means

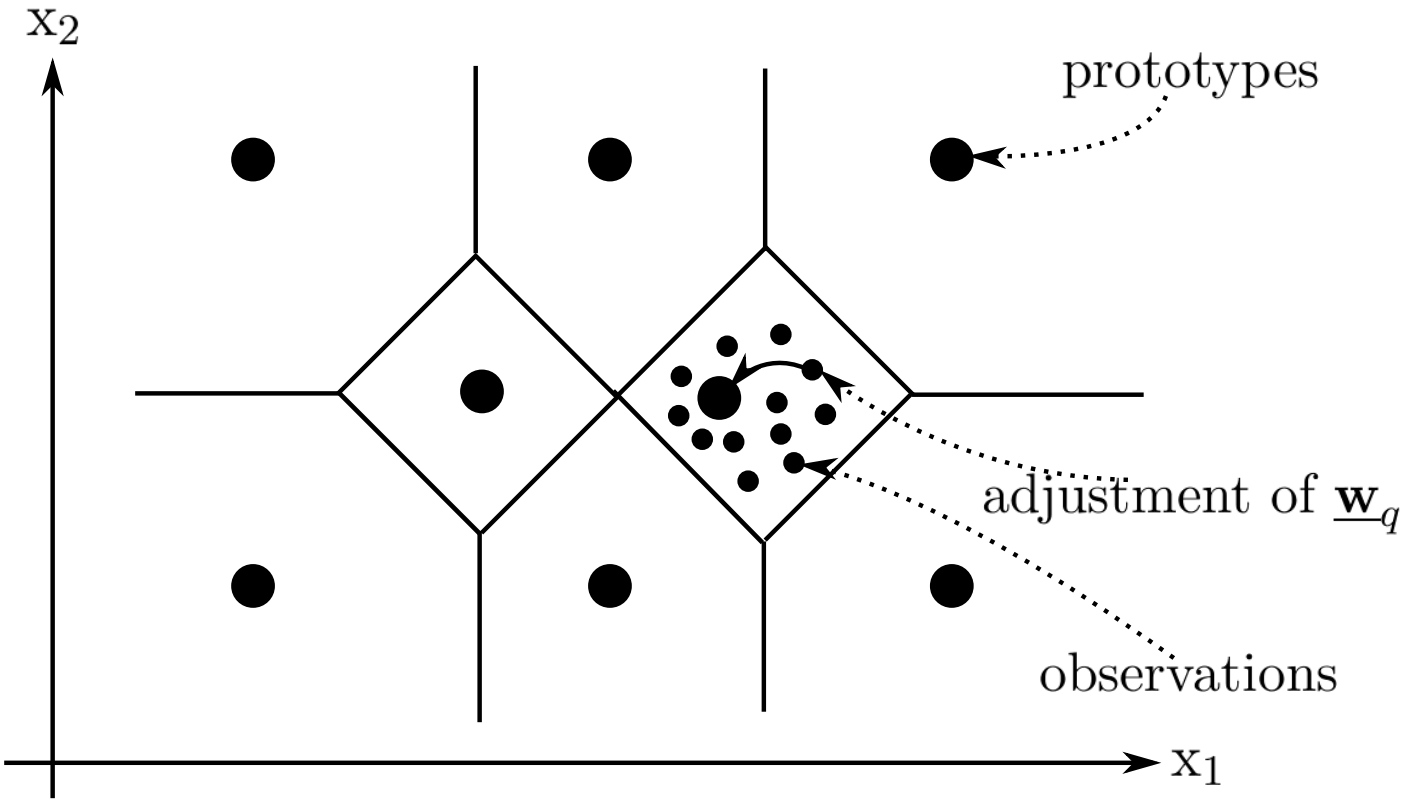
- prototypes: $\underline{\mathbf{t}}_q, q = 1, \ldots, M$
- binary assignment: $m_q^{(\alpha)} = 1$ if $\underline{\mathbf{x}}^{(\alpha)}$ belongs to cluster q, 0 else
- clustering **cost function**: $E\big[\{m_q^{(\alpha)}\}, \{\underline{\mathbf{t}}_q\}\big] = \frac{1}{2p} \sum_{q,\alpha} m_q^{(\alpha)} \|\underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{t}}_q\|^2$



Batch K-means: algorithm

Initialization of $\underline{\mathbf{t}}_q, q = 1, \ldots, M$ (e.g.around data's center of mass)
BEGIN loop

1. assign every data point to its nearest prototype
   $m_q^{(\alpha)} = 1$ if $q = \mathrm{argmin}_\gamma \left|\underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{t}}_\gamma\right| \qquad 0$, else
2. choose $\underline{\mathbf{t}}_q$ such that E^T is minimal (for the given -new- assignments)
   $$\underline{\mathbf{t}}_q = \frac{\sum_\alpha m_q^{(\alpha)} \underline{\mathbf{x}}^{(\alpha)}}{\sum_\alpha m_q^{(\alpha)}} \qquad \text{(center of mass of its assigned data)}$$

END loop

Clustering: illustration